

Overview - Classifiers

How are the image encoded in a CNN ?

There are two types of images:

- A grayscale image = one matrix (2D array)
- A color image = three matrices stacked together (3D array)

We will focus on the color image.

The image is converted into a 3D tensor of shape Width*Height*Channels – Channels = 1 for a grayscale image, and Channels = 3 for a color image where the 3 stands for RGB (Red, Green, Blue), which is composed of 3 superposed matrix – one matrix for each RGB color.

These matrix are filled with numbers between 0 and 255 to measure light intensity emitted or captured in that color channel – 0 for no intensity and 255 for max intensity.

A pixel (picture element) is the smallest measurable component in an image. Each pixel's value encodes how bright or colorful that tiny portion of the image is.

Thus, when we look at one pixel of the image, 3 values characterize that pixel – one for each color RGB. Thanks to the 3 values of RGB colors, the color of the pixel is determined.

As an example, here are many possible values for the RGB-vector and the interpretation:

- (255, 0, 0): bright red
- (128, 0, 0): dim red
- (0, 255, 0): pure green
- (0, 0, 255): pure blue
- (0, 0, 0): pure black
- (255, 255, 255): pure white
- (255,255,0): bright red + bright green = yellow
- (128,128,128): all RGB colors half-on = medium gray

How do the CNN manage to interpret these values?

The CNN is composed of filters = kernels (e.g first layer has 32 filters, second layer has 64...), and each kernel is independent and produces its own output, which is a 2D matrix representing a feature map showing where the pattern occurs. All kernels scan the image in parallel, producing multiple feature maps that are stacked together as the output of the layer.

When the kernel slides, we call it "Convolution".

Each kernel has 3 RGB sub-kernels – one for each RGB color (KR, KG, KB), plus a bias b. Let's first simplify with only one pixel, and then we'll generalize: for each color, each RGB sub-kernel attributes a value of importance between -1 and +1 dedicated to the color they focus on.

Each kernel is trained so that its weights adjust to detect a useful pattern with a specific feature like edges in different directions, color patterns, textures or corners.

Even though there are an infinite number of possible kernels, the CNN does not try all infinite kernels, it just uses the learned subset that is most useful for classification or detection. It makes a choice on which filters are more accurate and more appropriate for that image.

For instance, here is a possible kernel:

- (KR, KG, KB, b) = (1, -1, 0, 4)
- KR = 1 : the kernel “likes” high red intensity
- KG = -1 : the kernel “dislikes” green intensity
- KB = 0 : the kernel doesn’t care about blue intensity
- b = 4 > 0 : increases the activity of the kernel

Here is another one:

- (KR, KG, KB, b) = (0.1, -0.85, 0.7, -2)
- KR = 0.1 : the kernel is more likely to ignore blue intensity
- KG = -0.85 : the kernel is more likely to “dislike” green intensity
- KB = 0.7 : the kernel is more likely to “like” high red intensity
- b = -2 < 0 : decreases the activity of the kernel

Also, the CNN standardizes the RGB values of the image to scale them up between 0 and 1 with:

$$x' = \frac{x}{255}$$

Then, for each pixel, each kernel applies the same output formula to produce its own feature map.

$$Y(i, j) = \sum_{x=0}^{\text{Height}-1} \sum_{y=0}^{\text{Width}-1} \sum_{z=0}^{\text{Channels}-1} K(x, y, z) \cdot X(i + x, j + y, z) + b$$

Where:

- Y(i,j) = output of the convolution at position (i,j)
- K = kernel
- X = input image
- b = biais

Let’s take the example of a single pixel to see if our kernel activates strongly or not on that pixel:

- The kernel : (KR, KG, KB, b) = (1, -1, 0, 5)
- The pixel : (R,G,B) = (200,50,100)
- The output (we don’t standardize for clarity) : $Y = 1*200 + (-1)*50 + 0*100 + 5 = 155 > 0$
- The output is positive, so the kernel is very active

Here is another example :

- The kernel : (KR, KG, KB, b) = (0.1, -0.85, 0.7, -2)
- The pixel (we don’t standardize for clarity) : (R, G, B) = (100, 200, 0)
- The output : $Y = 0.1*100 + (-0.85)*200 + 0.7*0 + (-2) = -162 < 0$
- The output is negative, so the kernel is less active

Then, this calculation is repeated for every valid position (i,j) of the kernel on the image, and all these scalar outputs are then arranged spatially into a feature map (a 2D matrix).

Let's note that each kernel can look at more than one pixel at the same time, it can take a small grid as input, and thus return a grid or just a number as an output.

How is the activation function used in a CNN and what is its purpose ?

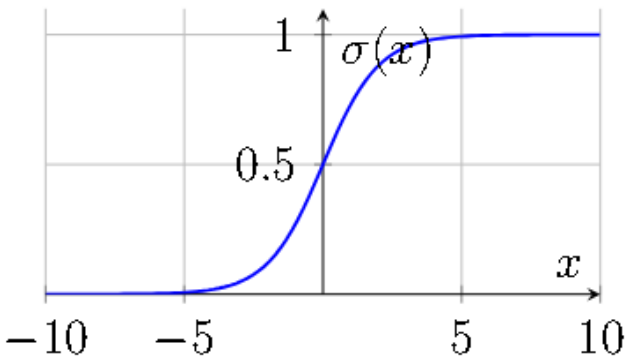
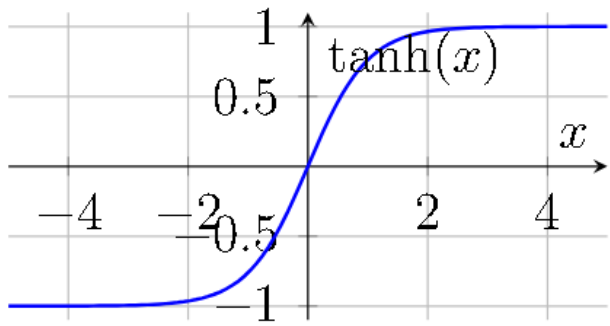
Once the full feature map Y has been computed, it is passed through an activation function — usually element-wise.

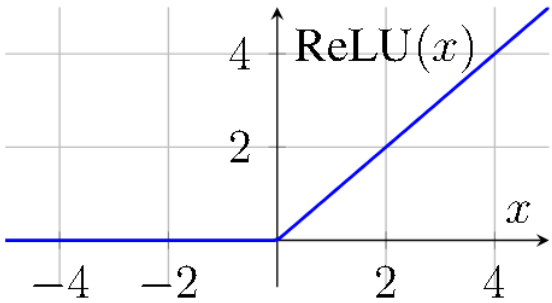
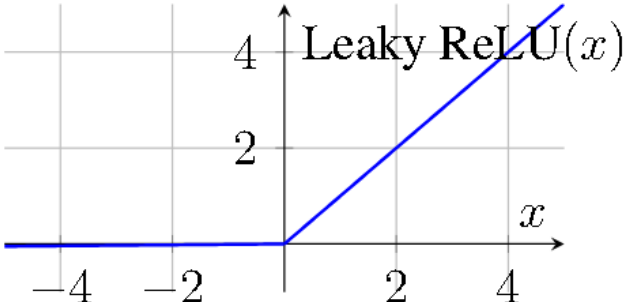
$$Z(i,j) = \varphi(Y(i,j))$$

Where φ is a nonlinear function

Without an activation function, when the CNN stacks multiple linear layers, it's still linear, so the network can only model linear relationships between inputs and outputs, which is not enough for complex data like images. The activation function introduces nonlinearity, allowing the CNN to learn complex and nonlinear features like edges, shapes, textures, object parts, etc.

There are different kind of activation functions, there is:

The sigmoid $\sigma(x) = \frac{1}{1 + e^{-x}}$ Output range : [0,1] Usage : - Maps any input to the range [0,1] - Extreme values are close to 0 or 1	
The TanH (Hyperbolic tangent) $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$ Output range: [-1,1] Usage: - Similar to sigmoid, but any input is mapped and centered in [-1,1] - Balances positive and negative signals.	

The ReLU (Rectified Linear Unit) $\text{ReLU}(x) = \max(0, x)$ Output Range: $[0, +\infty)$ Usage: - Gradient = 1 for positive values - Gradient = 0 for negative values	
Leaky ReLU $\text{Leaky ReLU}(x) = \begin{cases} x, & x \geq 0 \\ \alpha x, & x < 0 \end{cases}$ Output range : $(-\infty, \infty)$ Usage : - Like ReLU but gives a small negative slope when $x < 0$ - Prevents neurons from dying	

The ReLU is dominant for feature extraction – detecting which color is very bright and important.

How does the CNN goes from analyzing each pixel of the image to detecting what the image is about?

The raw pixel values (R,G,B) are transformed layer by layer into increasingly meaningful representations.

Layer	What the pixel values represent
Input layer	Physical light intensities (0–255 per channel)
Conv Layer 1	Weighted combinations of R/G/B values : edge maps, texture maps
Conv Layer 3–5	Abstract features : curves, corners, blobs
Deep layers	Semantic parts : “dog ear”, “wheel”, “face”
Final layer	Class logits : “dog”, “car”

To sum up, based on features, the CNN predicts a class label.